

Internal Working of HashMap

HashMap initial size is 16 → Can store Employee objects as Keys and Roles as Values.

Interview Q&A

```
Map<Employee, String> map=new HashMap<>();
```

6



HashMap uses hash function for **PUT** Operation

Hash output for e1 → 6, Bucket is Empty simply places the element inside bucket 6.

```
Map<Employee, String> map=new HashMap<>();
```

```
Employee e1=new Employee(101,"Basant")
```

```
Employee e2=new Employee(102,"peter")
```

```
Employee e3=new Employee(103,"John")
```

```
Employee e4=new Employee(104,"Sham")
```

```
map.put(e1,"Dev")
```

```
put(K k, V v)
    hash(k)
    index = hash & (n-1)
```

6



Hash output for e2 → 9, Bucket is Empty simply places the element inside bucket 9.

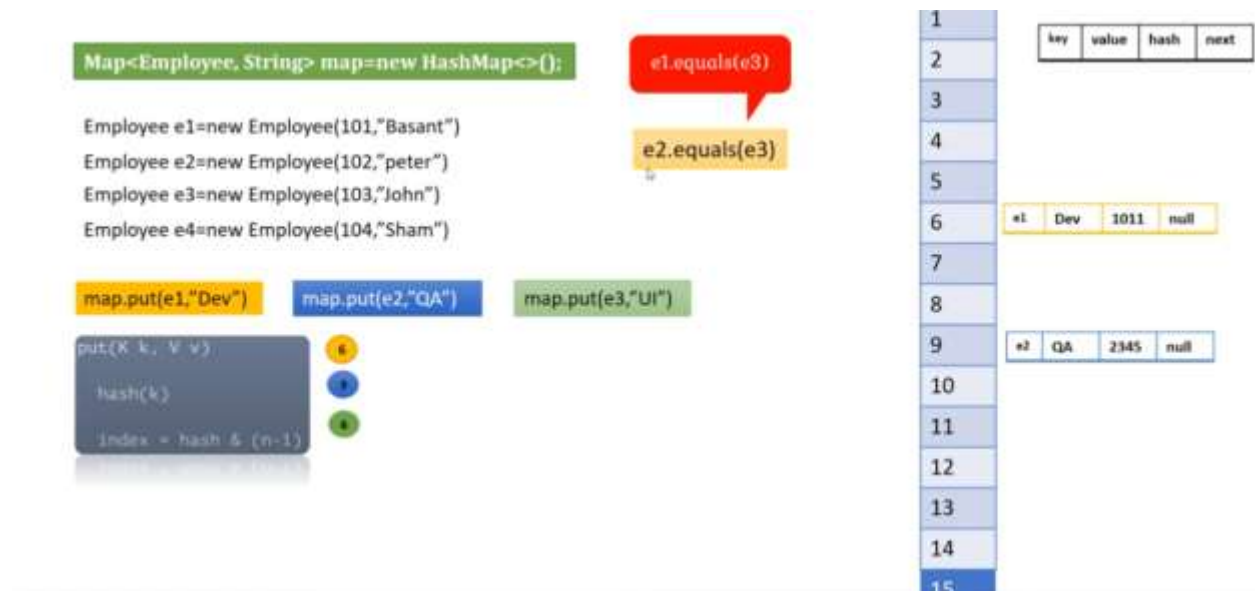


Hash output for e3 → 6, Bucket is not Empty. E1 is present inside bucket 6.

Hasing Collision

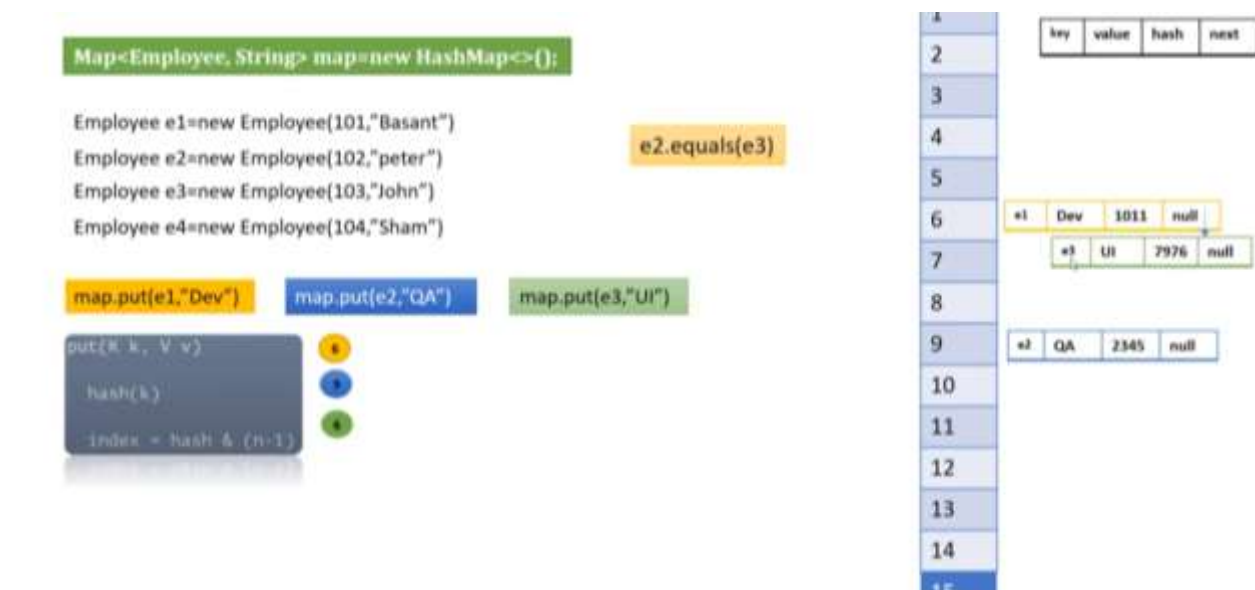


Since e1,e3 has same hash value, and e1 is already in bucket 6, we can't simply place e3 into bucket 6. To resolve this collision, it internally check **equals()** method → to check both the entries content wise equal or not.

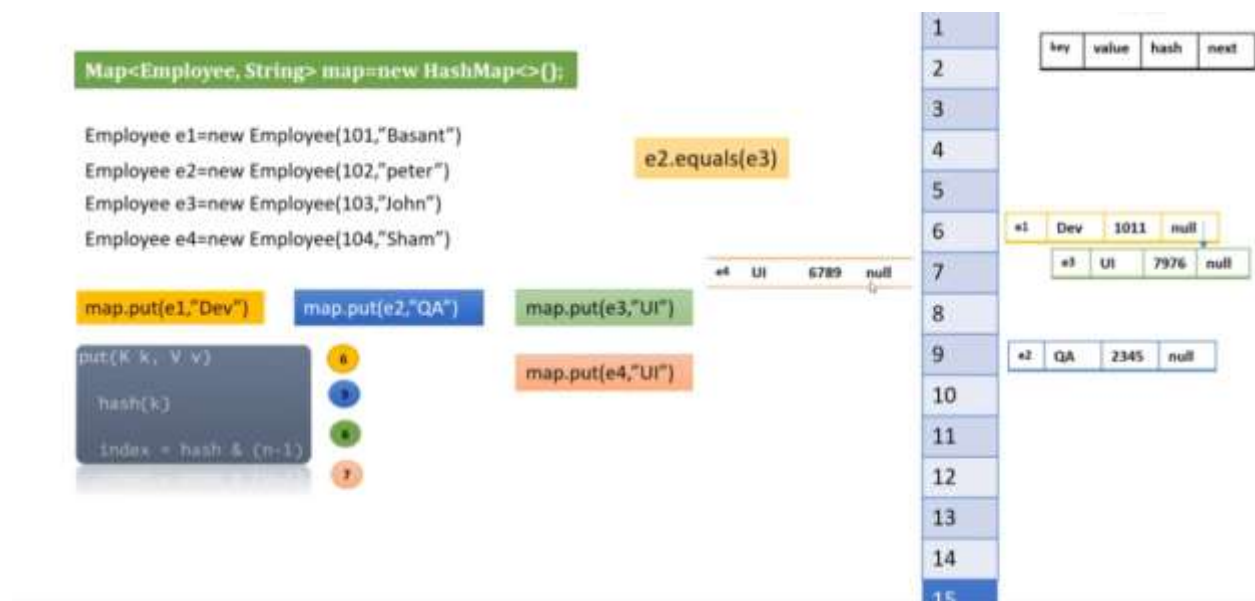


Since, content wise e1 and e3 are not same, so e3 will also be in bucket 6, next to e1.

Placed in bucket 6. e1 → e3 (LinkedList)



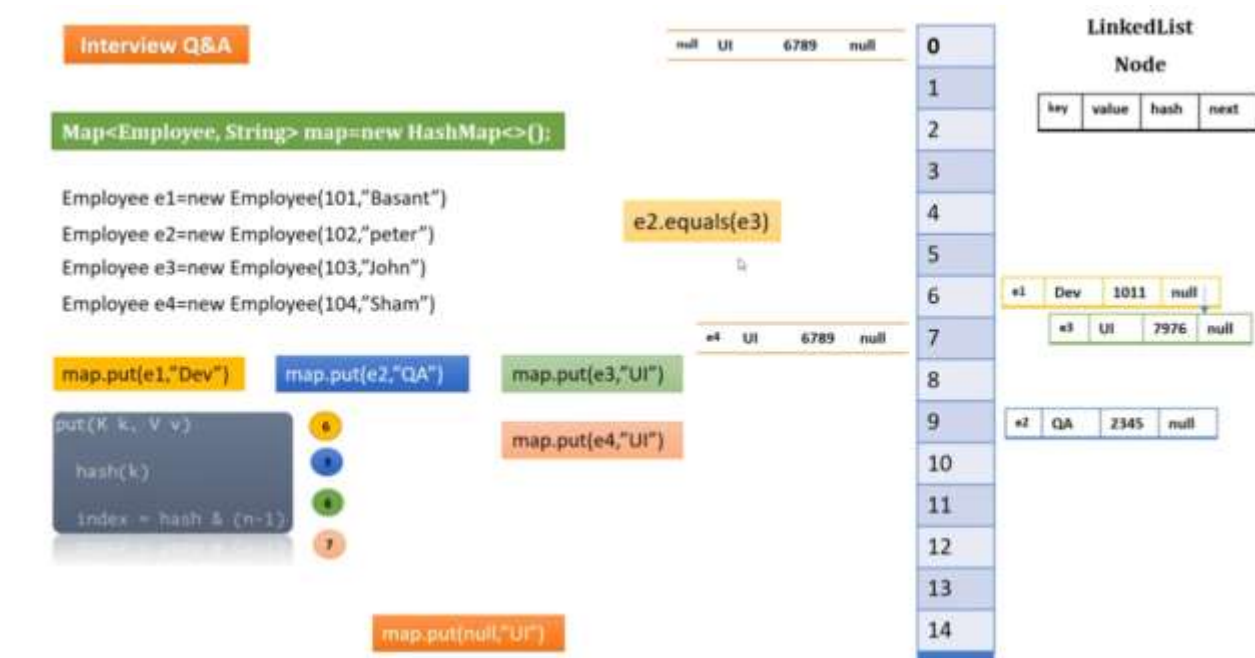
E4 → hash function output comes out as 7. Placed inside bucket 7.



1. If interviewer asks you, to put a (**null,"UI"**) Key value pair, with Key null value.

Answer should be straightforward for this tricky question

Because, Null (key) cannot be hashed (hash function) → entry will be placed in **0** bucket



2. What is the Enhancement done in HashMap in java8?

HashMap Enhancement in Java 8

Before Java 8:

- HashMap used an **array of buckets**
- Each bucket stored elements using a **Linked List**
- In worst case (many collisions), time complexity became **$O(n)$**

After Java 8:

- When the number of elements in a bucket exceeds a threshold (**8**), the Linked List is converted into a **Balanced Tree (Red-Black Tree)**
- When elements reduce below threshold (**6**), it converts back to **Linked List**

Why this improvement?

To improve performance in high-collision scenarios.

Time Complexity Comparison

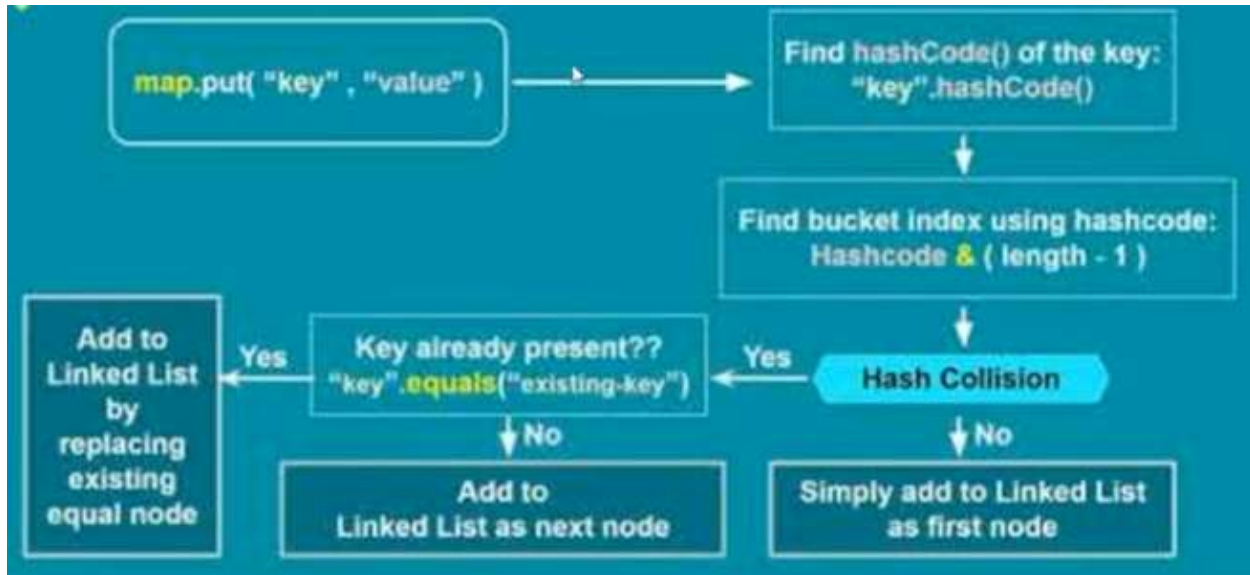
Operation	Before Java 8	After Java 8
Average	$O(1)$	$O(1)$
Worst	$O(n)$	$O(\log n)$

Key Points

- Uses **Red-Black Tree** for better search efficiency
- Treeification happens only when:
 - Bucket size ≥ 8
 - AND total capacity ≥ 64
- Otherwise, it stays as Linked List

“In Java 8, HashMap improves worst-case performance by converting collision chains from Linked List to Red-Black Tree, reducing time complexity from $O(n)$ to $O(\log n)$.”

HashMap Internal Working – Top Interview Questions



1. How does HashMap store data internally?

Uses **array of buckets**

Each bucket stores:

- Linked List (before Java 8)
- Balanced Tree (after Java 8, high collisions)

2. What happens during put() operation?

1. Calculate `hashCode()`
2. Convert to index (bucket)
3. If bucket empty → insert
4. If collision:
 - Check `equals()`
 - If same → replace value
 - Else → add to chain/tree

3. What happens during get() operation?

1. Calculate hashCode()
2. Go to bucket
3. Traverse:
 - Use equals() to find exact key

4. Why equals() and hashCode() both needed?

- hashCode() → find bucket
- equals() → find exact key

5. What is collision?

Multiple keys mapping to same bucket

6. How does Java 8 improve collision handling?

Converts Linked List → **Red-Black Tree** (Upon reaching threshold of 8, converts back if <6)

Reduces worst-case:

- $O(n) \rightarrow O(\log n)$

7. What is load factor?

Default = **0.75**

- Determines when to resize
- $\text{capacity} * \text{loadFactor}$

8. What is rehashing?

When threshold exceeded:

- Capacity doubles
- All elements rehashed

9. Why initial capacity is power of 2?

For efficient index calculation:

$\text{index} = \text{hash} \& (\text{n} - 1)$

10. Can HashMap have null keys/values?

- One null key 
- Multiple null values 

11. Difference between HashMap and Hashtable?

- HashMap → not synchronized
- Hashtable → synchronized

12. What happens if hashCode is same but equals is false?

- Both stored in same bucket
- No issue (handled via chaining/tree)

13. What happens if equals is true but hashCode is different?

- Contract violation
- Leads to unexpected behavior (duplicate keys)

14. Is HashMap thread-safe?

No

Use:

- `ConcurrentHashMap`
- `Collections.synchronizedMap()`

15. Why HashMap is fast?

Because:

- Direct bucket access ($O(1)$)
- Minimal traversal

16. What is Treeification?

Converting Linked List → Tree when:

- Bucket size ≥ 8
- Capacity ≥ 64

17. What is De-treeification?

Tree → Linked List when size drops below 6

18. How iteration works internally?

Traverses:

- Bucket array
- Then nodes inside each bucket

19. Why resizing is expensive?

Because:

- All elements are rehashed
- Time complexity becomes $O(n)$

20. Can we use mutable objects as keys?

Dangerous

If key changes → hashCode changes → retrieval fails

“HashMap uses hashing with buckets, handles collisions via chaining/tree, and relies on hashCode() and equals() for efficient $O(1)$ average operations.”